Ex 2.2.4: a scouting round before the anchored-op experiments

> Scouting pass over a more varied set of operations, on the grid alone, so that the removal reads of the anchored-op experiments are easier to interpret. The hue, saturation, and value ops spread their answers through the color cube, and they are where the answer moves furthest once the red operand loses its red; but a small change in the red operand reaches their answer no more often than it does for the current ops, and they are the first ops where operand order matters. Three commutative ops spread and are sensitive to both operands: `difference`, `exclusion`, and `mix` done in HSV. We propose a table with all six added and `add` dropped.

Observations

Each line below is a read on the grid, together with the statistic it rests on. None of them is a result. The next preregistration will adopt what it needs from here and check it on trained models.

- The op set: every candidate spreads its answers more evenly than the current ops do. A one-level change in the red operand reaches the answer on 61% to 100% of red lines for the commutative candidates and on 37% to 55% for the one-attribute HSV ops, the range `screen` and `multiply` sit in. Zeroing the red operand's R runs the other way: the answer moves furthest on the HSV trio. We propose table A+: drop `add`, add `difference`, `exclusion`, and `hsvmix`, and carry `hue-hsv`, `sat-hsv`, and `value-hsv` as a marked subset, the first ops in the grammar that read operand order.
- The one-level rule of E8 responds to rounding as much as to the op: `mix` counts as dependent on 61% of its own red probe lines and 49% of the shared draw. Under stochastic rounding the rule is graded and reads 0.50 on both sets, and no op moves by as much as 0.1, so the ranking of the ops is the same either way.
- What we make of it: the removal statistic should be a distance rather than exact match, scored on lines where zeroing the red operand's R moves the answer far, and the grammar should adopt stochastic rounding and the whole-line labeller from the two pilots, for comparability with M3.

How to read this

This is the scouting round planned in the backlog item: one notebook, the cheapest version of each question, no gates and no verdicts. Each section says what we ran, shows one figure or table, and says whether it changes the design. Sections land as their questions are run, and this one covers the op set. The tied-readout diagnostic and the τ against λ_a trade are still to come. The answer-drawing labeller ran as a pilot of its own, beside a pilot of stochastic rounding, and the closing section reads both.

Ex-2.2.3 anchored *red* on a grammar of six operations, then measured removal: does the model still answer correctly once the *red* axis is projected out of the residual stream?[1] Removal came out partial on four of the six ops. E8 traced that to the ops themselves: on the saturating ops, most red lines have an answer that would be the same even if the red operand were a little less red, so a model that has lost *red* can still answer them.

The diverse-op todo item proposes adding ops whose answers spread through the cube and depend on both operands, such as the hue, saturation, and brightness blend modes, alongside some of the saturating ones, so that the removal tests have both kinds to read.

The op-set

What we see. We took nine candidates ops,each defined on the same 256-color grid and mapped to the way the ink on top of it (0–15), and ran the whole 256 ink-at-answer in the resulting table.

Three of them work channel-by-channel and are commutative: `XOR(Palette, ink)`,`AND(Palette, ink)`,and `OR(Palette, ink)`.Three more are commutative but not channel-by-channel: `saturate_add`, `average`, and `max`. The other six take one attribute from the color-operand and the rest from the first; the two, `saturate_sub` and `luminosity-modulate of the HSV`,completely opin, as in Photoshop, and the same three in HSV played, as in Krita.

op	ink	commutative	graph	visible	how many
`XOR`	0–15	yes	16	16	256
`AND`	0–15	yes	16	16	256
`OR`	0–15	yes	16	16	256
`saturate_add`	0–15	yes	16	16	256
`average`	0–15	yes	16	16	256
`max`	0–15	yes	16	16	256
`saturate_sub`	0–15	no	256	16	256
`luminosity-modulate of the HSV`	0–15	no	256	16	256
`hue-set`	0–15	no	256	16	256
`sat-set`	0–15	no	256	16	256
`value-lock`	0–15	no	256	16	256

The Where-do. *The six ops of a 256-color grid, each mapped to the ink-at-answer table. Commutative is the mean of considered pairs whose answer is the same in both operand orders. A dash means the op is not commutative without counting. Bolded cells mark cases the color whose answer is stable: that color operand, the color the table always returns.*

Where the answers land

Each is the answer-cloud from the 2,5 extended to the candidates. Each panel shows the color cube seen from the gray diagonal, with one of the six ink-colors as the answer. Reading the answer there: no op that spreads its answers puts small masks everywhere. A saturating op piles them onto a face, an edge, or a corner.

*Where the answers of each op land. One mask per gray-axis view, one panel/mask in the middle column. Each panel shows the answer-cloud that color. All answers share one scale: a mask that likes its gold gets bolder. In HSV-space, each HSV-cube

Method

Everything in this section is computed on the grid when the notebook runs, using `sca.data.ops`: the six ops of ex-2.2.3 as `OPS` and the nine candidates as `CANDIDATES`, both snapped to the 216-color grid by the same `snap`.

Lines. There are 46,656 ordered pairs of grid colors, each one an (op1, op2) line, and 23,436 unordered pairs, which we use where a statistic is symmetric. A *red line* has dose ≥ 0.8, where dose is the larger of the two operand rednesses, as in ex-2.2.3. The shared probe draw is the one from ex-2.2.3: every color as op1 against 27 partners drawn once from the grid, with the same partners for every op other than `mix`. The probe set for `mix` is the 5,832 closed pairs from D2.1.

Spread is the Shannon entropy, in bits, of the distribution of answers over the 216 grid colors, taken over unordered pairs. **Channel marginals** are the share of unordered pairs whose answer has each grid level in each channel.

Dependence follows E8. For a red line, take the redder operand, lower its R by one grid level (3) or to zero, leave everything else alone, and ask whether the snapped answer changes. We report it as a share of the lines in the set named. Under **stochastic rounding** the answer of a line is the distribution `answer_dist` gives, the product over channels of a two-point distribution on the levels either side of the raw value, and the read is the total variation distance between the distributions of the line and of its lowered version, averaged over the set. **Moves by** is the Euclidean distance in the unit cube between the snapped answers of the line and of its lowered version, averaged the same way. **Reads op1 / op2** replaces that operand with a color drawn uniformly from the grid (seed 0, one draw per line, every fifth line) and asks whether the answer changes.

Commutative is the share of unordered pairs on which the op gives the same answer in both orders. `hsvmix` falls short of 1 on the pairs whose hues are opposite, where the circular mean is undefined and the rule keeps the hue of op1.

Op-relevance is `relevance` from the same module, now over an arbitrary table: for each line, the number of other ops in the table that give the same answer as the anchored op. Ex-2.2.3 counted unordered pairs; the tables here count ordered pairs, so that the ops that read operand order are counted on both orders.

---

1. The *residual stream* is the running vector of activations that each transformer layer reads from and writes back to; it is where we place the anchor. ↩

2. Entropy in bits, so a higher number means the answers are spread over more colors. ↩

3. Stochastic rounding is the grammar variant of the stochastic-rounding pilot: a raw channel value between two grid levels rounds to the upper one with probability equal to how far along it sits, drawn once per line at corpus build. The answer of a line is then a distribution over up to eight colors, and `answer_dist` in `sca.data.ops` computes it. The share of mass that moves between two distributions is their total variation distance. ↩